Day 168 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 1 reference

Watch: Hyperparameters Tuning: Grid Search vs Random Search (DataMListic) | IML17: Grid vs random search for hyperparameter tuning? (Bevan Smith)

`day168_grid_vs_random_search.py`, segment by segment, in the order `main()` uses things. Today's new parts are to_value, grid_unit, random_unit, decode, best_so_far and importance, copied unchanged into day168_exercise.py; the model is Day 109's boosting, the data and split Day 161's, and log loss Day 165's. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: u = a number in [0, 1) · U = trials as rows of the unit cube · k = values per setting in a grid · d = settings · A, V, F = training, tuning and fresh rows · G, P = the grids and the random pool, each (points, configurations, results)

Segment 0 — Setup

Imports and constants

CODE

```
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day161_exercise as d161
    import day165_exercise as d165
SPACE = (("learning rate", "log", 0.01, 1.0),
         ("depth", "int", 1, 8),
         ("min leaf", "logint", 1, 300))
ROUNDS = 100
N_TUNE = 6000
POOL = 200
BUDGETS = (8, 27, 64)
SHIFTS = 5
REPEATS = 1000
```

- `with contextlib.redirect_stdout(io.StringIO()):` — each exercise prints its hand checks when imported; this sends that text to a string nobody reads.
- `SPACE = (("learning rate", "log", 0.01, 1.0), ...)` — a tuple of tuples: name, scale, lowest and highest value of each hyperparameter.
- `BUDGETS = (8, 27, 64)` — 2^3, 3^3, 4^3: budgets a 3-setting grid can match exactly.

Segment 1 — The parts

to_value: three scales

MATH

$log: lo\ (hi/lo)^u \quad int: floor(lo + u(hi - lo + 1)) \quad logint: floor(lo\ ((hi + 1)/lo)^u)$

CODE

```
def to_value(u, kind, lo, hi):
    if kind == "log":
        return float(lo * (hi / lo) ** u)
    if kind == "int":
        return int(min(hi, np.floor(lo + u * (hi - lo + 1))))
    return int(min(hi, np.floor(np.exp(np.log(lo) + u * (np.log(hi + 1) - np.log(lo))))))
```

- `lo * (hi / lo) ** u` — u = 0 gives lo, u = 0.5 the geometric middle (0.1 for 0.01 to 1), u near 1 gives hi.
- `np.floor(lo + u * (hi - lo + 1))` — hi – lo + 1 equal slices of [0, 1), one per whole number.
- `min(hi, ...)` — a guard: u is below 1, but rounding could still reach hi + 1.
- `np.log(hi + 1)` — the log-spaced whole numbers stretch to hi + 1 so that hi itself gets a full slice.

grid_unit, random_unit, decode and best_so_far

CODE

```
def grid_unit(k, d, shift=None):
    s = np.zeros(d) if shift is None else np.asarray(shift, float)
    axes = [(np.arange(k) + 0.5 + s[j]) / k for j in range(d)]
    return np.array(list(itertools.product(*axes)))
def random_unit(n, d, rng):
    return rng.rand(n, d)
def decode(U, space=SPACE):
    return [{nm: to_value(u, kind, lo, hi) for u, (nm, kind, lo, hi) in zip(row, space)} for row
        in U]
def best_so_far(scores):
    return np.minimum.accumulate(np.asarray(scores, float))
```

- `np.zeros(d) if shift is None else np.asarray(shift, float)` — a conditional expression: no shift unless one is given.
- `(np.arange(k) + 0.5 + s[j]) / k` — the centres of k equal slices, moved by s[j] of a slice; a list comprehension builds one array per axis.
- `itertools.product(*axes)` — the star unpacks the list into separate arguments; product yields every combination, k^d of them.
- `rng.rand(n, d)` — an n × d array of uniform numbers from the seeded generator.
- `{nm: to_value(...) for u, (nm, kind, lo, hi) in zip(row, space)}` — a dictionary comprehension; the tuple in the loop target unpacks each SPACE entry.
- `np.minimum.accumulate` — the running minimum: the best score after each trial.

importance: eta squared

MATH

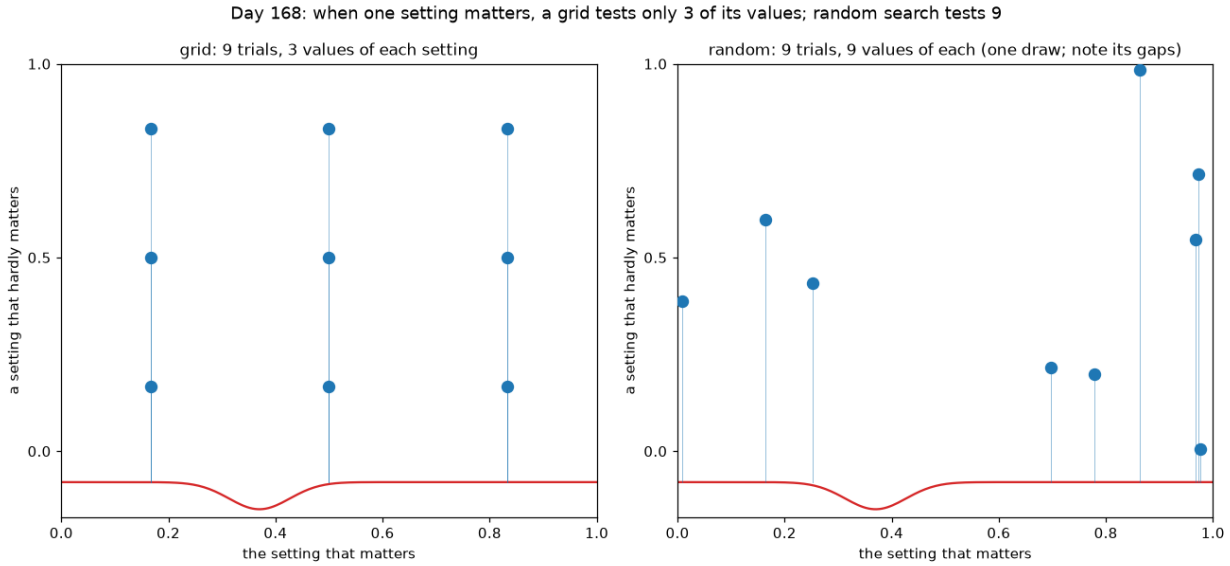
$$\eta^2 = \Sigma_b\, n_b\, (m_b - m)^2 \,/\, \Sigma_i\, (s_i - m)^2$$

CODE

```
def importance(U, scores, bins=5):
    s = np.asarray(scores, float); tot = np.sum((s - s.mean()) ** 2)
    out = []
    for j in range(U.shape[1]):
        b = np.minimum((U[:, j] * bins).astype(int), bins - 1)
        m = np.array([s[b == i].mean() if np.any(b == i) else 0.0 for i in range(bins)])
        n = np.bincount(b, minlength=bins)
        out.append(float(np.sum(n * (m - s.mean()) ** 2) / tot))
    return np.array(out)
```

- `(U[:, j] * bins).astype(int)` — which of the equal slices each trial's value falls in; np.minimum keeps u near 1 in the last slice.
- `s[b == i].mean() if np.any(b == i) else 0.0` — a boolean mask selects one slice's scores; an empty slice gets 0 (its count is 0, so it adds nothing).
- `np.bincount(b, minlength=bins)` — how many trials fall in each slice, with zeros for empty ones.

SYSTEM DIAGRAM



Nine trials as a grid and at random.

Segment 2 — The experiments

demo_benchmark: a new hidden optimum for each problem

MATH

$$score(p) = \Sigma_j\, w_j\, (p_j - c_j)^2$$

CODE

```
rng = np.random.RandomState(0); out = {}
for name, w in (("one of three matters", np.array([1.0, 0.01, 0.01])), ("all three matter
    equally", np.ones(3))):
    for n in BUDGETS:
        k = round(n ** (1 / 3)); G = grid_unit(k, 3)
        rg, rr = [], []
        for _ in range(REPEATS):
            c = rng.rand(3)
            f = lambda P: np.sum(w * (P - c) ** 2, 1)
            rg.append(f(G).min()); rr.append(f(random_unit(n, 3, rng)).min())
        rg, rr = np.array(rg), np.array(rr); out[(name, n)] = (rg.mean(), rr.mean())
        print(f"  {name:25s}, {n:2d} trials: mean shortfall grid {rg.mean():.5f}, random
            {rr.mean():.5f}; "
              f"random better in {np.mean(rr < rg):.3f} of {REPEATS} problems")
```

- `k = round(n ** (1 / 3))` — the cube root: 64 trials means 4 values per setting.
- `f = lambda P: np.sum(w * (P - c) ** 2, 1)` — a one-line function of all trials at once; broadcasting subtracts c from every row.
- `rg.append(f(G).min())` — the best of the grid's trials for this problem.
- `out[(name, n)] = ...` — a tuple as a dictionary key: one entry per kind of problem and budget.
- `np.mean(rr < rg)` — the mean of a boolean array is the share of True: how often random search won.

evaluate: one fit per configuration

CODE

```
def evaluate(configs, X, y, A, V, F):
    res = []
    with np.errstate(over="ignore"):
        for c in configs:
            t0 = time.time()
            _, E = d109.fit_boost(X[A], y[A], "log", ROUNDS, c["learning rate"], c["depth"],
                evals=(X[V], X[F]), min_leaf=c["min leaf"])
            res.append((d165.log_loss(y[V], d109.sigmoid(E[0][-1])), d165.log_loss(y[F],
                d109.sigmoid(E[1][-1])), time.time() - t0))
    return np.array(res)
```

- `with np.errstate(over="ignore"):` — a context manager: inside it, numpy does not warn when exp overflows (sigmoid's answer, 0 or 1, is still right).
- `evals=(X[V], X[F])` — Day 109's boosting returns the log-odds on each extra matrix after every round; [-1] is after the last tree.
- `res.append((..., ..., time.time() - t0))` — a tuple of three: tuning loss, fresh loss and seconds; np.array turns the list into a table.

demo_real: grids, a pool, and 1,000 random orders

CODE

```
t0 = time.time(); G = {}
for n in BUDGETS:
    U = grid_unit(round(n ** (1 / 3)), 3); G[n] = (U, decode(U), evaluate(decode(U), X, y, A, V,
        F))
    vals = [sorted({c[nm] for c in G[n][1]}) for nm, _, _, _ in SPACE]
    b = int(np.argmin(G[n][2][:, 0]))
    print(f"  grid of {n:2d}: values {vals}; best log loss {G[n][2][b, 0]:.4f} at {G[n][1][b]}")
U = random_unit(POOL, 3, np.random.RandomState(1)); P = (U, decode(U), evaluate(decode(U), X, y,
    A, V, F))
print(f"  ({POOL} random configurations and {sum(BUDGETS)} grid points evaluated in {time.time() -
    t0:.0f} s)")
b = np.argsort(P[2][:, 0])
for i in b[:3]:
    print(f"  pool's best: log loss {P[2][i, 0]:.4f} at " + ", ".join(f"{k} {v:.3g}" for k, v in
        P[1][i].items()))
print(f"  pool's worst: log loss {P[2][b[-1], 0]:.4f} at " + ", ".join(f"{k} {v:.3g}" for k, v in
    P[1][b[-1]].items()))
rng = np.random.RandomState(2); orders = [rng.permutation(POOL) for _ in range(REPEATS)]
S = {}
for n in BUDGETS:
    best = np.array([P[2][o[:n], 0].min() for o in orders])
    g = G[n][2][:, 0].min()
    print(f"  {n:2d} trials: grid {g:.4f}; random median {np.median(best):.4f} (2.5% to 97.5%:
        {np.quantile(best, 0.025):.4f} to "
          f"{np.quantile(best, 0.975):.4f}); random at least as good in {np.mean(best <= g):.3f}
              of {REPEATS} searches")
S["random"] = np.array([best_so_far(P[2][o[:max(BUDGETS)], 0]) for o in orders])
S["grid"] = np.array([best_so_far(G[max(BUDGETS)][2][o, 0]) for o in
    (rng.permutation(max(BUDGETS)) for _ in range(REPEATS))])
sh = np.random.RandomState(5).rand(SHIFTS, 3) - 0.5
sg = [evaluate(decode(grid_unit(3, 3, s)), X, y, A, V, F)[:, 0].min() for s in sh]
best27 = np.array([P[2][o[:27], 0].min() for o in orders])
print(f"  {SHIFTS} shifted grids of 27: best " + ", ".join(f"{v:.4f}" for v in sg) + f" (mean
    {np.mean(sg):.4f}; the centred grid {G[27][2][:, 0].min():.4f}); "
      f"random of 27 at least as good as a shifted grid in {np.mean([np.mean(best27 <= v) for v in
          sg]):.3f} of comparisons")
```

- `G[n] = (U, decode(U), evaluate(...))` — each grid stored as points, configurations and results.
- `sorted({c[nm] for c in G[n][1]})` — a set comprehension removes repeats; sorted lists the values a grid tried.
- `[rng.permutation(POOL) for _ in range(REPEATS)]` — 1,000 random orders of the pool: each is one random search.
- `P[2][o[:n], 0].min()` — fancy indexing: the first n configurations of an order, their tuning losses, the best.
- `np.quantile(best, 0.025)` — the 2.5% point of the 1,000 searches' results.
- `np.random.RandomState(5).rand(SHIFTS, 3) - 0.5` — random shifts between –0.5 and 0.5 of a slice, one row per shifted grid.

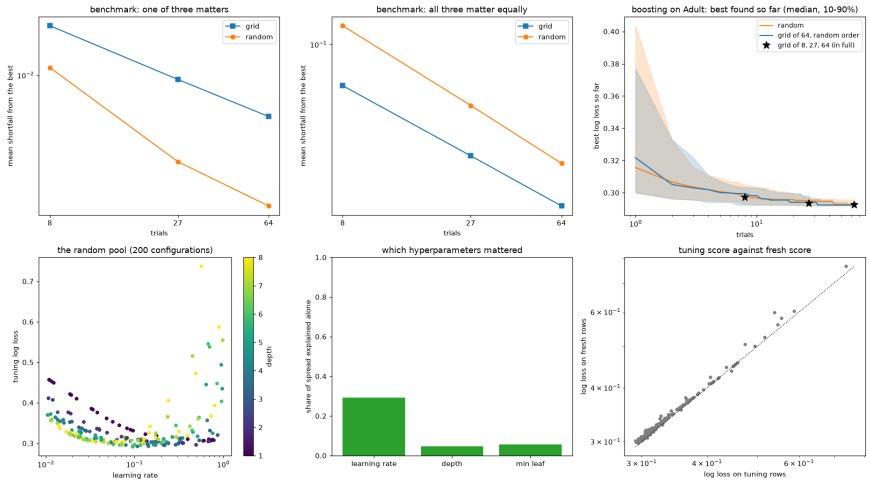
demo_importance and demo_fresh: a pair of settings, and ranks

CODE

```
s = P[2][:, 0]; c = np.minimum((P[0][:, 0] * 4).astype(int), 3) * 4 + np.minimum((P[0][:, 1] *
    4).astype(int), 3)
m = np.array([s[c == i].mean() for i in range(16)]); n = np.bincount(c, minlength=16)
print(f"  learning rate and depth together (4 x 4 cells): explain {np.sum(n * (m - s.mean()) ** 2)
    / np.sum((s - s.mean()) ** 2):.3f}")
i = int(np.argmin(tune)); r = np.argsort(np.argsort(fresh))[i] + 1
```

- `np.minimum(..., 3) * 4 + np.minimum(..., 3)` — two slice numbers from 0 to 3 combined into one cell number from 0 to 15.
- `np.argsort(np.argsort(fresh))` — argsort twice gives each item's rank (0 = best); + 1 counts from 1.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `itertools.product(*lists)` — every combination of one item from each list.
- `f(*args)` — unpacks a list into separate arguments.
- `x if cond else y` — a conditional expression.
- `np.minimum.accumulate(a)` — the running minimum.
- `np.errstate(over="ignore")` — silences one kind of numpy warning inside a with block.
- `np.argsort(np.argsort(a))` — the rank of each item.

End of Day 168 syntax reference. For the experiments and what they showed, see the Day 168 complete guide.